

IoT and Artificial Intelligence Integration (AIoT)

A Case Study on AI-Enabled Remote Patient Monitoring

Project	Task 4: Mini Project – IoT Case Study (CodeAlpha IoT Internship)
Prepared by	Yuvan Sundar J

Abstract

The Internet of Things (IoT) connects sensors, actuators and embedded devices to networks so that the physical world can be measured and controlled remotely. On its own, however, IoT mostly collects data; it does not understand it. Artificial Intelligence (AI), and machine learning (ML) in particular, supplies that missing understanding by finding patterns, detecting anomalies and predicting what will happen next. The combination is commonly called **AIoT**. This case study explains why AI and IoT are being integrated, describes a typical AIoT architecture and its enabling technologies, and then works through a practical example: an **AI-enabled remote patient monitoring system** built on an ESP32 microcontroller, wearable sensors and cloud analytics.

1. Introduction

A modern IoT deployment can generate an enormous amount of data. A single wearable that samples a pulse signal at 100 Hz produces about 8.6 million readings per day, and a hospital ward, a factory or a city has thousands of such devices. Traditional IoT systems handle this with fixed rules, such as “raise an alarm if temperature exceeds a limit”, and with dashboards that a person must watch. This approach has clear limits:

- **Static thresholds** ignore individual differences and context, which leads to false alarms or missed events.
- **Sending everything to the cloud** costs bandwidth and energy and adds delay to time-critical decisions.
- **Human monitoring does not scale** when thousands of data streams arrive continuously.

AI addresses these limits: models learn what is normal for each device or person, flag deviations automatically and can run on low-power hardware near the sensor. This case study explains AIoT concepts, presents a layered architecture, analyses a healthcare use case in detail and discusses challenges and future scope.

2. From IoT to AIoT: What AI Adds

AI does not replace the IoT stack; it adds a decision-making layer on top of it. Table 1 summarises the main capabilities and typical techniques.

Capability	Typical techniques	IoT example
Classification	SVM, Random Forest, CNN	Activity recognition from accelerometer data
Anomaly detection	Z-score, Isolation Forest, autoencoder	Irregular heart rate; abnormal machine vibration
Forecasting	Regression, ARIMA, LSTM	Predicting energy demand or vital-sign trends
Optimisation	Reinforcement learning, fuzzy logic	Adaptive air-conditioning or irrigation

Table 1: Capabilities that AI adds to IoT systems

3. AloT System Architecture

An AloT system is usually organised into five layers, shown in Figure 1. Data flows left to right, while retrained models flow back to the devices.

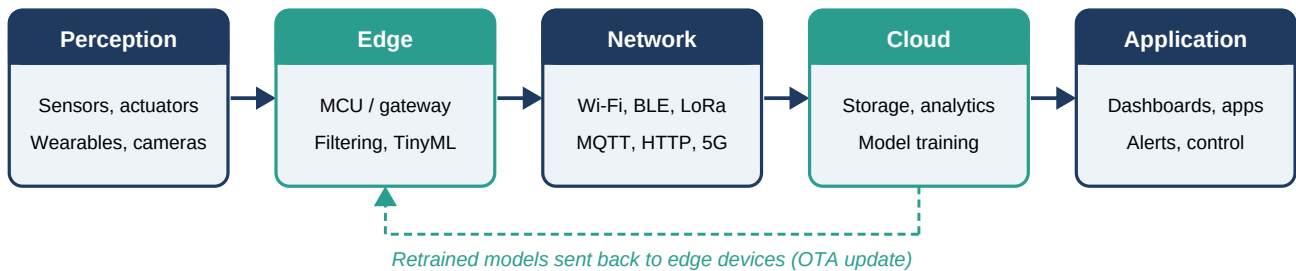


Figure 1: Layered AloT architecture

- **Perception layer:** sensors convert physical quantities (heart rate, temperature, motion, gas concentration) into electrical signals.
- **Edge layer:** a microcontroller or gateway filters the data, extracts features and can run a small ML model locally.
- **Network layer:** carries data using protocols suited to the range and power budget, such as BLE for wearables, Wi-Fi for indoor devices and LoRa for long range. MQTT is the most common lightweight messaging protocol.
- **Cloud layer:** stores history, trains larger models on data from many devices and provides long-term analytics.
- **Application layer:** presents results to users through dashboards and mobile apps, and triggers alerts or control actions.

3.1 Edge AI versus Cloud AI

A key design decision is where the ML model runs. Table 2 compares the two options. In practice most systems use a **hybrid** approach: fast, simple inference at the edge and heavy training in the cloud.

Factor	Cloud AI	Edge AI
Latency	Higher, because of the network round trip	Very low, decisions are made locally
Bandwidth	High, raw data is uploaded	Low, only events or summaries are sent
Privacy	Raw data leaves the device	Raw data can stay on the device
Compute and memory	Effectively unlimited	Very limited (kilobytes of RAM on a microcontroller)
Connectivity	Needs a reliable connection	Keeps working offline
Best suited for	Training, long-term trends, fleet-wide analytics	Real-time inference and immediate alerts

Table 2: Cloud AI compared with Edge AI

4. Key Enabling Technologies

- **TinyML:** techniques such as quantisation and pruning shrink neural networks so they fit on microcontrollers. TensorFlow Lite for Microcontrollers and Edge Impulse are widely used toolchains.
- **Data preprocessing:** filtering noise, normalising values, splitting the signal into sliding windows and extracting features are usually more important to accuracy than the choice of model.
- **Connectivity and protocols:** MQTT for publish/subscribe messaging, BLE and Wi-Fi for short range, LoRaWAN and NB-IoT for long range and low power, TLS for encryption.

- **Cloud IoT platforms:** AWS IoT Core and Azure IoT Hub, along with open tools such as ThingSpeak, InfluxDB and Grafana, provide device management, storage and visualisation.
- **Low-power hardware:** ESP32, ARM Cortex-M based boards and dedicated AI accelerators make on-device inference affordable.

5. Case Study: AI-Enabled Remote Patient Monitoring

5.1 Problem statement

Patients with chronic conditions, elderly people living alone and patients recovering at home need continuous observation, but hospital beds and clinical staff are limited. Periodic manual checks can miss sudden changes, and simple threshold alarms on wearables often trigger false alerts when a person exercises or moves. The goal is a low-cost system that monitors vital signs continuously, learns each patient's normal range and alerts a caregiver only when something genuinely unusual occurs.

5.2 Proposed system

The proposed reference design (Figure 2) uses an ESP32 as the wearable's brain. Sensors feed the ESP32, which cleans the signals and runs a lightweight anomaly check locally. Only summaries and flagged events are sent to a cloud broker over MQTT with TLS encryption. The cloud stores the data, learns a personal baseline for each patient and notifies the care team through a dashboard and mobile alerts.

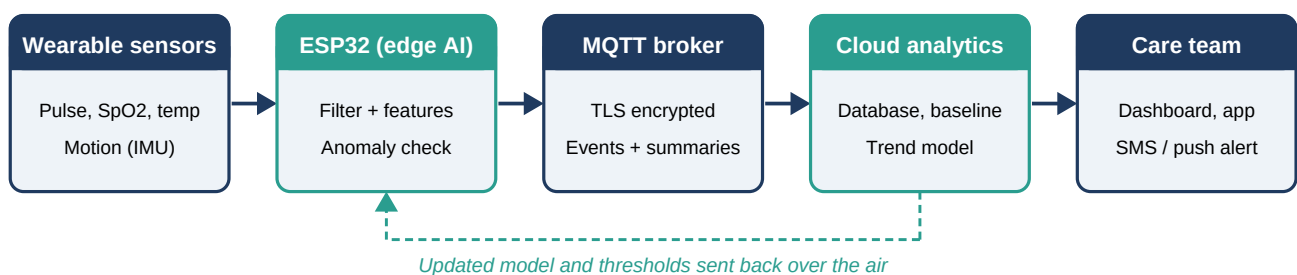


Figure 2: Data flow of the remote patient monitoring system

5.3 Hardware and software components

Component	Role in the system
ESP32 development board	Microcontroller with built-in Wi-Fi and BLE; runs preprocessing and the small ML model
MAX30102 sensor	Optical (photoplethysmography) sensor for heart rate and blood oxygen saturation (SpO2)
MAX30205 or DS18B20	Digital body-temperature sensor
MPU6050 (accelerometer and gyroscope)	Detects activity level and falls; helps separate exercise from a true abnormal reading
AD8232 (optional)	Single-lead ECG front end for rhythm analysis
Li-ion cell with charging module	Portable power supply
Mosquitto or AWS IoT Core	MQTT broker that receives messages from the device
InfluxDB and Grafana (or ThingSpeak)	Time-series storage and clinician dashboard

Table 3: Components of the reference design

5.4 AI pipeline

1. **Acquire:** sample the pulse signal and motion data at fixed rates (for example 100 Hz and 50 Hz).
2. **Preprocess:** apply a moving-average or band-pass filter, reject motion artefacts and normalise values.
3. **Extract features:** compute heart rate, SpO2, temperature and activity level over sliding windows (for example 30 seconds).
4. **Infer at the edge:** a compact model, or a rule plus statistics check, flags unusual windows. Critical conditions, such as sustained low SpO2, raise an immediate local alert without waiting for the cloud.
5. **Publish:** send only summaries and flagged events over MQTT with TLS, which saves bandwidth and battery.
6. **Analyse in the cloud:** build a personal baseline per patient, examine long-term trends and periodically retrain the model.
7. **Alert and update:** notify caregivers and push improved models or thresholds back to the device over the air (OTA).

5.5 Example: personalised anomaly detection

The snippet below shows the core idea in Python. Instead of a fixed limit, each reading is compared with the patient's own recent baseline using a rolling z-score. The same logic can be ported to C++ on the ESP32 or replaced by a trained model such as an Isolation Forest.

```
import numpy as np
from collections import deque

WINDOW = 60      # last 60 readings (about 1 minute at 1 Hz)
Z_LIMIT = 3.0    # flag readings 3 standard deviations from baseline
history = deque(maxlen=WINDOW)

def is_anomalous(hr):
    """True if this heart-rate reading deviates from the personal baseline."""
    if len(history) < WINDOW:      # still learning the baseline
        history.append(hr)
        return False
    mean, std = np.mean(history), np.std(history) or 1.0
    z = abs(hr - mean) / std
    history.append(hr)
    return z > Z_LIMIT
```

5.6 Expected benefits and evaluation

- **Earlier detection:** gradual changes in a patient's trend can be noticed before a fixed threshold is crossed.
- **Fewer false alarms:** using motion context and a personal baseline reduces unnecessary alerts.
- **Lower cost and energy use:** transmitting events instead of raw streams extends battery life and reduces data charges.
- **Privacy:** raw physiological signals can remain on the device.

The system should be evaluated using sensitivity (share of real events detected), false alarm rate, alert latency, battery life and comparison of sensor readings against a clinically approved device.

Note: this is a learning-oriented reference design. A system used for real patient care must be clinically validated and approved by the relevant medical device regulator (for example CDSCO in India or the FDA in the USA).

6. AloT in Other Domains

Domain	How AloT is used	Benefit
Smart agriculture	Soil-moisture and weather sensors with a model that decides when to irrigate	Saves water and improves yield
Industry (predictive maintenance)	Vibration and temperature sensors with anomaly detection on machines	Fewer breakdowns and less downtime
Smart homes and energy	Occupancy and usage patterns learned to control heating, cooling and lighting	Lower energy bills and better comfort
Smart cities	Edge vision on traffic cameras to adapt signal timing and detect incidents	Reduced congestion and faster response

Table 4: Examples of AloT applications

7. Challenges and Limitations

- **Security and privacy:** devices with default passwords, unencrypted links or unsigned firmware are easy targets, and ML models can be fooled by adversarial input. Health data is especially sensitive and is covered by laws such as GDPR, HIPAA and India's Digital Personal Data Protection Act, 2023. Mitigations include TLS, device authentication, secure boot, signed OTA updates and data minimisation.
- **Resource constraints:** microcontrollers have little memory, computing power and battery, so models must be compressed, which can reduce accuracy.
- **Data quality:** noisy sensors, motion artefacts, missing data and scarce labelled examples make training difficult.
- **Accuracy, bias and explainability:** a model trained on one group of people may perform poorly on another, and clinicians need to understand why an alert was raised.
- **Interoperability:** devices from different vendors use different formats and protocols, which complicates integration.
- **Regulation and ethics:** medical use requires validation, patient consent and clear accountability for automated decisions.

8. Future Scope

- **Wider TinyML adoption:** better tools and more efficient chips will bring richer models to coin-cell-powered devices.
- **Federated learning:** devices train models locally and share only model updates, improving privacy while still learning from many users.
- **5G and 6G:** low-latency, high-density connectivity will support real-time applications such as remote surgery assistance and connected vehicles.
- **Digital twins:** virtual replicas of patients, machines or cities, fed by live IoT data, will allow simulation and prediction before acting.
- **Generative AI interfaces:** language models can explain sensor readings in plain language and help clinicians or operators query system data.

9. Conclusion

AloT turns connected devices from simple data collectors into systems that can interpret their surroundings and act on them. The remote patient monitoring case study shows how a hybrid design, with fast inference at the edge and learning in the cloud, can give more personalised and timely alerts at low cost. Realising this potential depends on solving security, privacy, accuracy and regulatory challenges. As TinyML, federated learning and faster networks mature, AloT is likely to become a standard building block of healthcare, industry and smart cities.

References

- [1] OASIS, *MQTT Version 5.0 Specification*, OASIS Standard, 2019.
- [2] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, O'Reilly Media, 2019.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, 2016.
- [4] H. B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," *AISTATS*, 2017.
- [5] Espressif Systems, *ESP32 Technical Reference Manual*.
- [6] Maxim Integrated, *MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor Datasheet*.
- [7] TensorFlow, *TensorFlow Lite for Microcontrollers* documentation, tensorflow.org/lite/microcontrollers.